

A04-junit: Unit Testing

COMP 301

October 6, 2025

This assignment is a little different from the rest of the assignments in COMP 301. In most other assignments, you are given a description of a class or algorithm, and your job is to write code that implements the class. Most of your work inevitably goes into the `src/main` folder. In this assignment, you are still given a description of a class, but you are not required to write code that implements it. Instead, you are asked to write **unit tests** to verify if an arbitrary implementation of the class is correct. Since you'll be writing unit tests, most of your work will go in the `src/test` folder.

Each of Apprentice, Enchanter, and Sorcerer below describes a different class or classes for which you should write unit tests. When you submit your work to Gradescope, the autograder will run your unit tests on a series of different (hidden) implementations of the classes. Your tests must correctly identify which implementations are correct and which ones contain a bug.

Again, **you are not required to implement the classes described below for this assignment**. The goal of this assignment is to give you practice writing unit tests with JUnit 5.

1 Apprentice: Alphabetizer

The first class you should write tests for is called Alphabetizer. Write unit tests for Alphabetizer in the `src/test/java/com/comp301/a04junit/ApprenticeAlphabetizerTests` file.

An empty, unimplemented version of the class can be found in `src/main/java/com/comp301/a04junit/alphabetizer/Alphabetizer.java`. You should test this class file when writing unit tests. *This class file is the one you should test when writing unit tests.* The unit tests you write should verify that Alphabetizer matches the algorithm specification given below.

The Alphabetizer class is intended to sort an array of `String` values alphabetically and process the sorted strings one at a time.

1.1 Constructor

The Alphabetizer class has a constructor with the following signature:

```
public Alphabetizer(String[] arr)
```

When creating a new Alphabetizer instance, the constructor takes in an unsorted array of `String` values as an argument. The Alphabetizer instance keeps a reference to the provided array internally, and must **never modify or manipulate the array**.

1.2 next() method

Once an instance of Alphabetizer has been created, it can be used to process the elements in the array, one at a time, in alphabetized order. This can be accomplished by repeatedly calling the `next()` method. Each time `next()` is called, a different string from the array is returned. Strings are returned in alphabetized order.

After every string from the array has been returned once, `next()` should throw a `NoSuchElementException` if it is called again.

1.3 hasNext() method

Instead of relying on catching a `NoSuchElementException` to check if there are still remaining strings left to process, another option is to use the `hasNext()` method. `hasNext()` returns a boolean value which is `true` if there are still strings left, and `false` if there are no more strings left. In other words, `hasNext()` will only return `false` if the object is in a state where a call to `next()` would throw a `NoSuchElementException`.

Example Usage

Imagine a new Alphabetizer object is created for the array { "b1", "a1", "c1" }. Calling `next()` on the object for the first time will return "a1". The second time `next()` is called, it will return "b1"; and the third time it is called, it will return "c1". After the third time, any future calls to `next()` will throw a `NoSuchElementException`.

2 Enchanter: ItemImpl and PositionImpl

In Enchanter, you should write unit tests for the `ItemImpl` and `PositionImpl`. Write unit tests for `ItemImpl` in the `src/test/java/com/comp301/a04junit/EnchanterItemTests` file, and tests for `PositionImpl` in `src/test/java/com/comp301/a04junit/EnchanterPositionTests`.

Empty, unimplemented versions of these two classes can be found in:

`src/main/java/com/comp301/a04junit/adventure`

These class files are the ones you should be testing when writing unit tests.

A completed version of the `ItemImpl` class implements the following interface and contains an encapsulated `String` for the `name` field.

```
public interface Item {
    String getName();
    @Override
    boolean equals(Object other);
    @Override
    String toString();
}
```

The completed `PositionImpl` class implements the following interface and contains an encapsulated fields for `x` and `y`.

```
public interface Position {
    int getX();
    int getY();

    /**
     * Constructs and returns a new Position object located one unit in the indicted
     * direction from
     * the "this" object
     */
    Position getNeighbor(Direction direction);
}
```

And the `Direction` is an enumerated type:

```
public enum Direction {
    NORTH,
    SOUTH,
    EAST,
    WEST
}
```

3 Sorcerer: InventoryImpl and PlayerImpl

In Sorcerer, you should write unit tests for the `InventoryImpl` and `PlayerImpl` classes. Write unit tests for `InventoryImpl` in the

`src/test/java/com/comp301/a04junit/SorcererInventoryTests` file, and tests for `PlayerImpl` in

`src/test/java/com/comp301/a04junit/SorcererPlayerTests`.

Empty, unimplemented versions of these two classes can be found in

`src/main/java/com/comp301/a04junit/adventure`. *These class files are the ones you should be testing when writing unit tests.* You can see the required interfaces for the two classes below:

```

public interface Inventory {
    /** Returns true if the Inventory is empty */
    boolean isEmpty();

    /** Getter method for the number of Items currently stored in the inventory */
    int getNumItems();

    /** Getter method which returns a cloned list of the Items in the Inventory */
    List<Item> getItems();

    /** Adds an Item to the Inventory */
    void addItem(Item item);

    /** Removes an Item from the Inventory */
    void removeItem(Item item);

    /** Clears the Inventory */
    void clear();

    /** Removes the Items from an "other" Inventory, and adds them into "this" Inventory */
    void transferFrom(Inventory other);
}

```

```

public interface Player {
    Position getPosition();
    Inventory getInventory();
    String getName();
    /**
     * Blindly moves the player's position one unit in the indicated direction, without
     * checking
     * whether the new location is valid
     */
    void move(Direction direction);
}

```

4 Points and Grading

- **Apprentice:** The unit tests you write for the `Alphabetizer` class are worth a total of 4 points.
- **Enchanter:** The unit tests you write for the `ItemImpl` and `PositionImpl` classes are worth 4 points each, for a total of 8 points.
- **Sorcerer:** The unit tests you write for the `InventoryImpl` and `PlayerImpl` classes are worth 4 points each, for a total of 8 points.
- **Style:** Conforming to the Google style guide is worth 1 point.

The total possible points is 21.

Important note about grading: When calculating your grade percent for this assignment, a point value of 11 or less will be considered a 0% on the assignment, while a grade of 21 will be considered a 100%. Any grade between 11 and 21 will be linearly scaled between 0% and 100%. You can calculate your grade percent with the following formula:

$$\text{Grade Percent} = \max\left(\frac{p - 11}{10}, 0\right) \times 100\%$$

where p is the number of points that the autograder gave you.